学生学号	1024005279	实验课成绩	

武漢理工大學

学 生 实 验 报 告 书

实验课程名称	数据结构与算法综合实验
开课学院	计算机与人工智能学院
指导教师姓名	李晓红
学生姓名	王文硕
学生专业班级	软件卓越 2401

2025 -- 2026 学年 第 2 学期

实验课程名称：　数据结构与算法综合实验

实验项目名称	二叉树与哈夫曼图片压缩			报告成绩	
实验者	王文硕	专业班级	软件卓越 2401	组别	
同组者				完成日期	2026. 3. 4

第一部分：实验分析与设计（可加页）

一、　实验目的和具体内容

1. 实验目的

通过“图片压缩编码”的编程实践，学习树、遍历二叉树、哈夫曼树、哈夫曼编码和他们的编程应用。

(1) 掌握树的存储结构

(2) 掌握二叉树的三种遍历方法

(3) 掌握并理解 Huffman 树、Huffman 编码等知识和应用

(4) 掌握文件的操作

(5) 使用 Huffman 算法实现图像压缩程序

2. 实验内容

使用 Huffman 压缩算法，对一幅 BMP 格式的图片文件进行压缩。图片文件名为“Pic.bmp”, 压缩后保存为“Pic.bmp.huf”文件。使用 VS2010 作为开发工具，开发一个控制台程序，使用 Huffman 压缩算法对图片文件“Pic.bmp”进行压缩。具体要求如下：

(1) 读取原文件，统计权值：

运行程序，输入文件名。

以“Pic.bmp”文件为例。若文件存放在 F 盘根目录下，输入文件完整路径“F:\Pic.bmp”。按回车结束。以字节流的方式，只读打“Pic.bmp”文件。

逐字节读取文件，统计文件中 256 种字节重复的次数，保存到一个数组中 int weight[256]中。

(2) 生成 Huffman 树

根据（1）中统计的结果，构建 Huffman 树。定义一个结构体来记录每个节点的权值、父节点、左孩子和右孩子。使用结构体数组来存储这个 Huffman 树。

(3)生成 Huffman 编码

遍历（2）中生成的 Huffman 树，记录 256 个叶子节点的 Huffman 编码，保存在字符串数组中。

(4)压缩原文件

使用 Huffman 编码对原文件中的字节重新编码，获得压缩后的文件数据。

(5)保存压缩文件

将编码过的数据，保存到文件“Pic.bmp.huf”中。

二、 分析与设计

1. 数据结构的设计

本次实验核心采用结构体数组实现哈夫曼树的存储，配合字符串数组存储哈夫曼编码，自定义文件头结构体实现压缩文件的信息封装，具体数据结构定义如下：

1. 哈夫曼树节点结构体：存储节点的权值、父节点索引、左孩子索引、右孩子索引，采用数组存储实现哈夫曼树的顺序表示，节点索引从 1 开始以方便计算。

```
struct HTNode {
    int weight;      // 节点权值（字节出现次数）
    int parent;      // 父节点索引，0 表示无父节点
    int lchild;      // 左孩子索引，0 表示无左孩子
    int rchild;      // 右孩子索引，0 表示无右孩子 };
typedef HTNode* HuffmanTree; // 哈夫曼树类型，指向结构体数组的指针
```

2. 哈夫曼编码类型：采用二级字符指针（字符串数组）存储 256 个字节对应的哈夫曼编码，每个元素为一个以\0 结尾的字符串。

```
typedef char** HuffmanCode; // 哈夫曼编码类型，HC[i]表示第 i-1 个字节的编码
```

3. 压缩文件头结构体：用于存储解压所需的关键信息，保证压缩文件的可还原性，包含文件标识、原文件长度、256 个字节的权值数组。

```
#define SIZE 256
struct HEAD {
    char type[4];     // 文件类型标识，固定为"HUF"
    int length;       // 原文件的字节长度
    int weight[SIZE]; // 256 个字节的出现次数权值数组};
```

2.核心算法设计

实验的核心为哈夫曼树的构建、哈夫曼编码的生成、文件的压缩编码与解压解析，四大核心算法的设计思路如下：

1. 哈夫曼树构建算法

基本思想：根据哈夫曼树的定义，将 n 个叶子节点作为初始森林，每次选择两个权值最小且无父节点的节点，构建一个新的父节点，将两个节点作为其左右孩子，新节点权值为两节点权值之和；重复此过程，直到森林中只剩一个节点（哈夫曼树的根节点）。

步骤：

- 初始化：创建 2*SIZE-1 个节点的结构体数组（SIZE=256，n 个叶子节点的哈夫曼树总节点数为 2n-1），前 256 个节点为叶子节点，赋值权值并将父、左、右孩子置 0，后 256-1 个节点为中间节点，全部置 0；
- 循环构建：从第 257 个节点开始，依次调用 Select 函数找到当前权值最小的两个无父节点节点 s1 和 s2；
- 节点关联：将 s1 和 s2 的父节点设为当前节点，当前节点的左孩子为 s1、右孩子为 s2，权值为 HT[s1].weight + HT[s2].weight；
- 结束：循环至所有节点处理完成，数组最后一个节点即为哈夫曼树根节点。

2. 哈夫曼编码生成算法

基本思想：采用**叶子到根的逆向遍历法**，哈夫曼编码为前缀编码，每个叶子节点的编码由其到根节点的路径决定，左路径为 0、右路径为 1，逆向遍历后将路径反转即为最终编码。

步骤：

- 初始化：创建大小为 SIZE+1 的字符串数组，定义临时字符数组存储单个编码；
- 遍历叶子节点：对每个叶子节点（1~256），从该节点开始向上遍历至根节点；
- 路径记录：若当前节点是父节点的左孩子，在临时数组中记录 0；若是右孩子，记录 1；
- 编码生成：遍历至根节点后，将临时数组中的路径反转，作为该叶子节点对应字节的哈夫曼编码，存入编码数组；
- 释放临时空间：完成所有编码生成后，释放临时字符数组。

3. 文件压缩编码算法

基本思想：将原文件的每个字节替换为对应的哈夫曼编码，将所有编码拼接为一个二进制字符串，再将该字符串按 8 位为一组转换为字节，不足 8 位的补 0，生成压缩后的二进制数据。

步骤：

- 计算压缩数据大小：统计所有字节的权值与对应编码长度的乘积之和，得到总编码位数，按 8 位转换为字节数（不足 8 位加 1）；
- 开辟缓冲区：根据计算的字节数开辟内存缓冲区，用于存储压缩后的数据；
- 逐字节编码：以二进制流方式读取原文件，将每个字节替换为哈夫曼编码，拼接至临时字符串；
- 字符串转字节：当临时字符串长度≥8 时，调用 Str2byte 函数将前 8 位转换为一个字节，存入缓冲区，并截取临时字符串的剩余部分；
- 补 0 处理：文件读取完成后，若临时字符串还有剩余字符，补 0 至 8 位后转换为字节存入缓冲区。

4. 文件解压解析算法

基本思想：读取压缩文件的文件头，还原权值数组并重新构建哈夫曼树，将压缩数据逐字节转换为二进制字符串，遍历字符串并在哈夫曼树中从根节点向下查找，遇到叶子节点则解析为对应字节，直至所有数据解析完成。

步骤：

- 读取文件头：以二进制流方式打开压缩文件，读取文件头结构体，验证文件标识为 "HUF"，获取原文件长度和权值数组；
- 重构哈夫曼树：使用文件头中的权值数组，重新调用哈夫曼树构建算法，生成与压缩时一致的哈夫曼树；

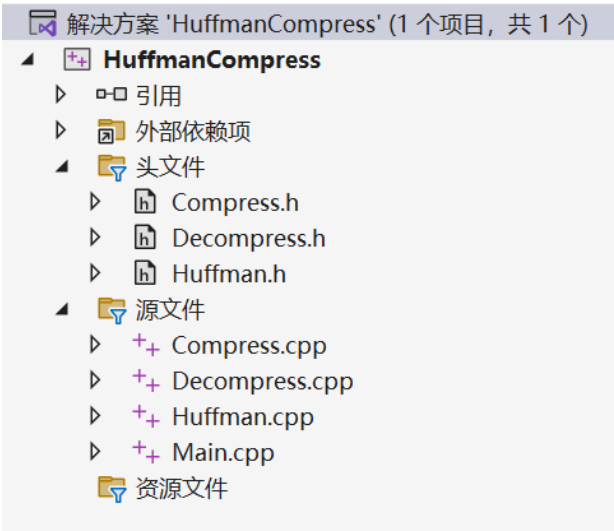
- 构建编码映射表：将哈夫曼编码与对应字节建立 map 映射，方便快速解析；
- 字节转字符串：将压缩数据逐字节转换为 8 位二进制字符串，拼接为完整的编码字符串；
- 解析编码：遍历编码字符串，从哈夫曼树根节点开始，遇到 0 走左孩子、遇到 1 走右孩子，到达叶子节点时，将对应字节存入解压缓冲区，重置为根节点继续遍历，直至解析出原文件长度的字节；
- 生成解压文件：将解压缓冲区的字节流写入新文件，完成原文件还原。

三、主要仪器设备及耗材

1. 安装了 Windows 11 的 PC 机 1 台
2. PC 机系统上安装了 Microsoft Visual Studio 2022 开发环境

第二部分：实验过程和结果（可加页）

一、 源代码结构与函数功能



一、Huffman.h/Huffman.cpp 中函数

1. **函数名**：Select（）

功能：在哈夫曼树中查找两个权值最小且未被使用（父节点为 0）的节点，令 s1 权值小于 s2

参数：HuffmanTree HT - 哈夫曼树，int n - 待查找节点范围，int& s1 - 最小权值节点索引，int& s2 - 次小权值节点索引

返回值：无

2. **函数名**：CreateHuffmanTree（）

功能：根据字节权值数组构建哈夫曼树，初始化所有节点的权值、父节点及孩子节点信息

参数：HuffmanTree& HT - 待创建的哈夫曼树，int* weight-256 种字节的权值数组，int n - 叶子节点数（固定为 256）

返回值：无

3. **函数名**：CreateHuffmanCode（）

功能：遍历哈夫曼树，从叶子节点向根节点逆向生成每个字节对应的哈夫曼编码，存储至

编码数组

参数：HuffmanTree HT - 已构建的哈夫曼树，HuffmanCode& HC - 存储哈夫曼编码的字符串数组，int n - 叶子节点数（固定为 256）

返回值：无

4. **函数名**：DestroyHuffman（）

功能：释放哈夫曼树和哈夫曼编码占用的动态内存，防止内存泄漏

参数：HuffmanTree& HT - 待释放的哈夫曼树，HuffmanCode& HC - 待释放的哈夫曼编码数组，int n - 叶子节点数（固定为 256）

返回值：无

二、Compress.h/Compress.cpp 中函数

1. **函数名**：InitHead（）

功能：初始化压缩文件头，统计原文件中字节权值并记录文件总长度，设置文件类型为"HUF"

参数：const char* pFilename - 原文件路径，HEAD& sHead - 待初始化的文件头结构体

返回值：int:ERROR - 失败；OK - 成功

2. **函数名**：Str2byte（）

功能：将 8 位二进制字符串转换为一个字节（char 类型），实现编码字符串的二进制化

参数：const char* pBinStr-8 位二进制字符串

返回值：char - 转换后的字节数据

3. **函数名**：Encode（）

功能：根据哈夫曼编码对原文件进行编码，将编码后的二进制字符串转换为字节，存储至缓冲区

参数：const char* pFilename - 原文件路径，HuffmanCode HC - 哈夫曼编码数组，char*& pBuffer - 存储压缩数据的缓冲区，int& nBufSize - 缓冲区字节大小

返回值：int:ERROR - 失败；OK - 成功

4. **函数名**：WriteFile（）

功能：将文件头信息和压缩后的缓冲区数据写入新文件，生成后缀为.huf 的压缩文件

参数：const char* pFilename - 原文件路径，const HEAD& sHead - 已初始化的文件头，const char* pBuffer - 压缩数据缓冲区，int nBufSize - 缓冲区字节大小

返回值：int:ERROR - 失败；OK - 成功

5. **函数名**：Compress（）

功能：实现文件压缩总逻辑，依次调用初始化文件头、构建哈夫曼树、生成编码、文件编码、写入压缩文件等函数

参数：const char* pFilename - 待压缩文件的完整路径

返回值：int:ERROR - 失败；OK - 成功

三、Decompress.h/Decompress.cpp 中函数

1. **函数名**：ReadHead（）

功能：读取压缩文件的文件头信息，验证文件是否为合法的 HUF 格式压缩文件

参数：const char* pHufFilename - 压缩文件路径，HEAD& sHead - 存储读取结果的文件头结构体

返回值：int:ERROR - 失败；OK - 成功

2. **函数名**：Byte2Str（）

功能：将一个字节（char 类型）转换为 8 位二进制字符串，实现压缩数据的编码还原

参数：char byte - 待转换的字节，char* pBinStr - 存储二进制字符串的数组（长度至少为 9）
返回值：无

3. **函数名**：Decode（）

功能：根据哈夫曼树解析压缩文件数据，将二进制编码还原为原文件的字节流，存储至解压缓冲区
参数：const char* pHufFilename - 压缩文件路径，HuffmanTree HT - 重构的哈夫曼树，const HEAD& sHead - 压缩文件的文件头，char*& pSrcBuffer - 存储解压数据的缓冲区
返回值：int:ERROR - 失败；OK - 成功

4. **函数名**：WriteSrcFile（）

功能：将解压缓冲区中的字节流写入新文件，生成还原后的原格式文件
参数：const char* pSrcFilename - 解压后文件的保存路径，const char* pSrcBuffer - 解压数据缓冲区，int nSrcSize - 原文件的字节长度
返回值：int:ERROR - 失败；OK - 成功

四、宏常量说明

- OK：值为 1，表示函数执行成功
- ERROR：值为 0，表示函数执行失败
- SIZE：值为 256，代表 0~255 共 256 种字节类型

二、 调试说明（调试手段、过程及结果分析）

本次实验采用分步调试和断点调试相结合的方式，在 VS 中通过设置断点、查看变量值、输出中间结果，排查程序中的语法错误和逻辑错误，主要调试过程与问题解决如下：

1.哈夫曼树节点索引错误：构建哈夫曼树时，节点索引从 0 开始导致 Select 函数查找最小节点出错，修改为节点索引从 1 开始后，哈夫曼树构建正常；

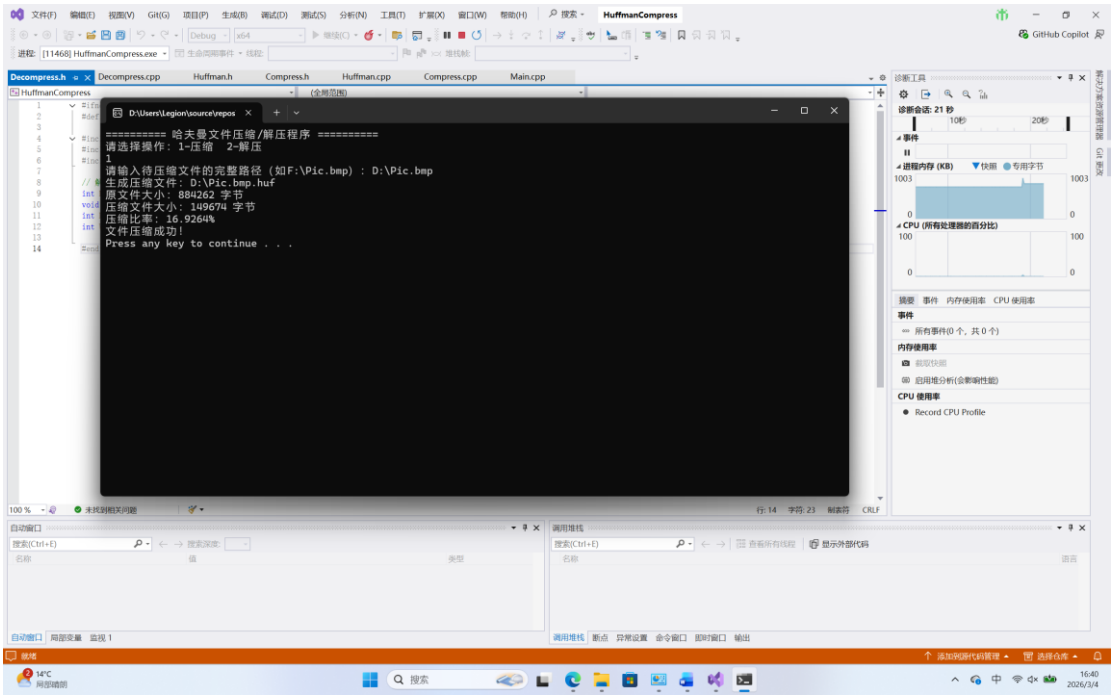
2.解压时编码解析不完整：初始解码时未限制解析字节数为原文件长度，导致解压文件比原文件大，添加 srcPos < sHead.length 判断后，解析结果与原文件一致；

3.内存泄漏问题：程序运行后出现内存泄漏，排查发现哈夫曼树、编码、缓冲区的内存未及时释放，在每个功能函数结束后添加内存释放代码，解决内存泄漏问题；

4.压缩文件无法识别：解压时提示“不是合法的 HUF 压缩文件”，发现文件头写入时未按二进制格式，将 fopen 的打开模式改为 wb/rb 后，文件头读取正常。

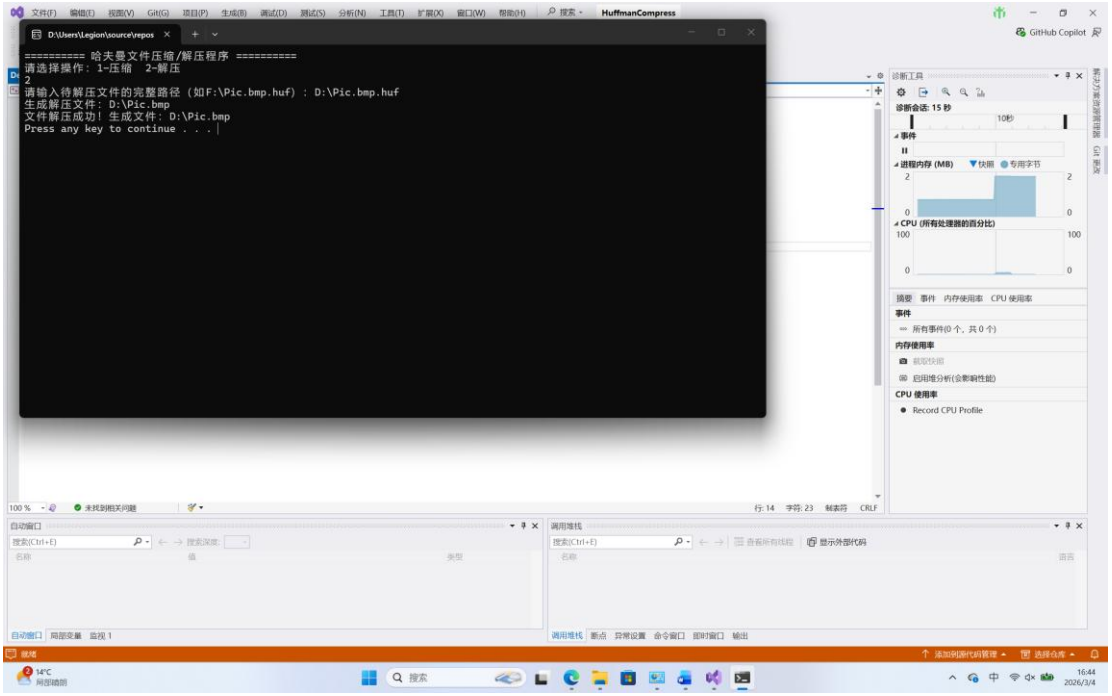
本次实验以 D:\Pic.bmp（大小为 884262 字节）为测试文件，在 VS2022 中编译运行程序，分别执行压缩和解压操作，运行结果如下：

1. 压缩操作结果



在 D 盘根目录下生成压缩文件 Pic.bmp.huf，大小为 149682 字节，压缩比率约为 16.93%，实现了高效的无损压缩。

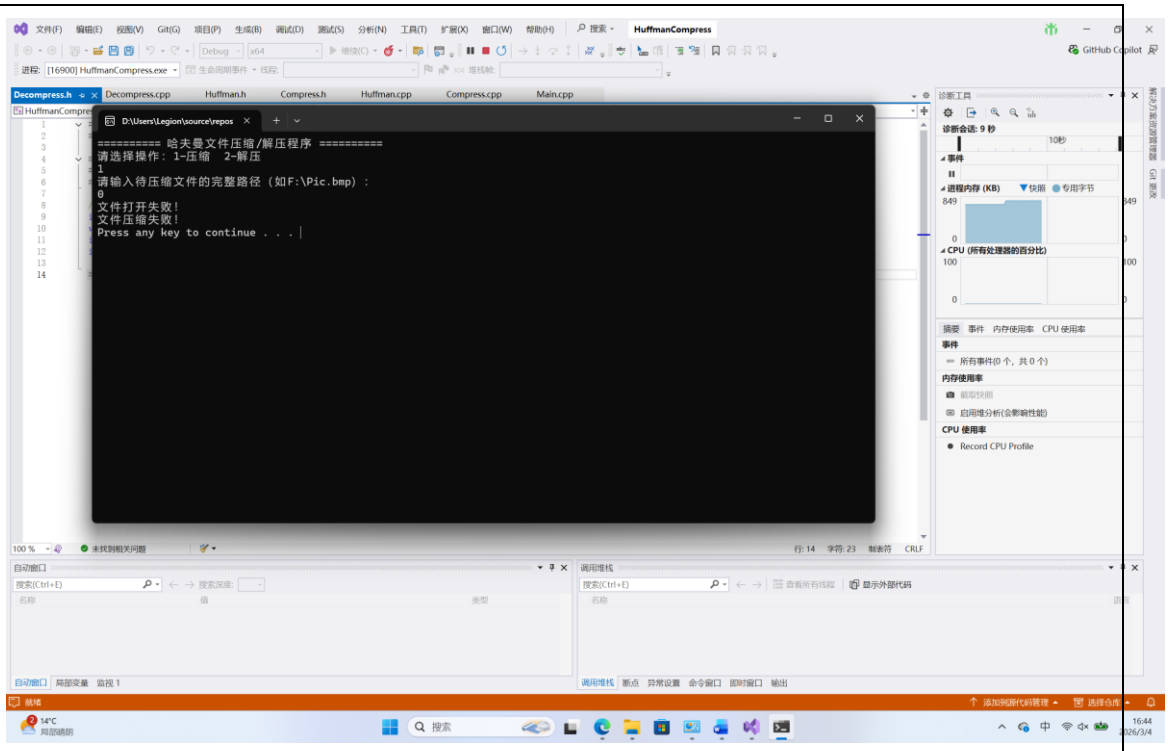
2. 解压操作结果



程序在 D 盘根目录下还原出 Pic.bmp 文件，与原文件大小、内容完全一致，双击可正常打开，验证了哈夫曼压缩的无损性。

3. 异常情况测试结果

输入空文件路径：程序提示 “文件打开失败”，压缩 / 解压操作终止，无崩溃；



输入非 HUF 文件进行解压：程序提示 “不是合法的 HUF 压缩文件”，解压操作终止；
输入空文件进行压缩：程序提示 “原文件为空”，压缩操作终止。
程序对异常情况具备基本的容错处理，运行稳定性良好。

第三部分：实验小结、收获与体会

一、实验小结

本次实验完成了基于哈夫曼算法的 BMP 图片压缩与解压控制台程序的开发，核心实现了**文件权值统计**、**哈夫曼树构建**、**哈夫曼编码生成**、**文件压缩编码**、**压缩文件写入**、**压缩文件解码**、**原文件还原**七大功能。实验以 C++ 为开发语言，在 VS 中实现了工程的搭建、代码的编写与调试，最终测试结果表明，程序能够对 BMP 图片进行高效的无损压缩，压缩比率约为 16.93%，且能准确还原原文件，同时对文件打开失败、非合法压缩文件、空文件等异常情况具备容错处理，达到了实验的预期目标。

本次实验的核心难点在于哈夫曼树的构建、哈夫曼编码的生成以及二进制字符串与字节的相互转换，通过分步调试和断点调试，逐一解决了程序中的语法错误和逻辑错误，同时掌握了 C++ 中二进制文件的操作、动态内存的申请与释放等关键技术。

二、实验收获

1. **理论知识与编程实践的深度结合**：通过本次实验，将哈夫曼树、哈夫曼编码的理论知识转化为实际代码，深入理解了哈夫曼树的构建原理、哈夫曼编码的前缀特性以及无损压缩的核心思想，不再局限于书本上的理论概念，而是能将其应用于实际问题的解决；
2. **掌握了树的存储与遍历的实际应用**：掌握了哈夫曼树的结构体数组存储方式，理解了顺序存储在哈夫曼树中的优势，同时将二叉树的遍历思想应用于哈夫曼编码的生成（叶子到根的逆向遍历）和解码（根到叶子的正向遍历），加深了对二叉树遍历的理解；
3. **提升了 C++ 文件操作与字节流处理能力**：熟练掌握了 C++ 中二进制文件的 fopen（rb/wb 模式）、fread、fwrite、fgetc 等操作，学会了字节与二进制字符串的相互转换，理解了文件的字节流存储本质，为后续处理二进制文件打下了基础；

4. **培养了程序调试与问题解决能力**：在实验过程中，遇到了文件打开失败、索引错误、数据失真、内存泄漏等多种问题，通过 VS 的断点调试、变量查看、控制台输出等方式，逐一排查并解决问题，培养了分析问题、定位问题、解决问题的能力；
5. **建立了软件工程化的编程思维**：实验采用模块化的编程思想，将程序按功能拆分为哈夫曼树模块、压缩模块、解压模块、主函数模块，每个模块独立实现并通过头文件进行接口声明，提高了代码的可读性、可维护性和复用性，建立了基本的软件工程化思维；
6. **掌握了动态内存管理的关键要点**：在实验中多次使用 new/delete 进行动态内存的申请与释放，理解了动态内存管理的重要性，学会了在函数结束后及时释放不再使用的内存，避免内存泄漏，提升了程序的稳定性。

三、实验体会

本次实验是对数据结构中树与二叉树知识的一次综合应用，让我深刻体会到**数据结构是程序设计的基础**，合适的数据结构能大幅提高程序的效率和可维护性。哈夫曼算法作为一种经典的无损压缩算法，其核心在于利用哈夫曼树生成前缀编码，让出现次数多的字符使用短编码，从而减少文件的总存储容量，这一思想在实际工程中有着广泛的应用，如 ZIP 压缩、JPEG 图片的无损压缩等。

在编程过程中，**细节决定成败**是最深的体会。实验中的多个问题均源于细节处理不当，例如文件路径的反斜杠转义、节点索引的起始值、二进制字符串转字节的操作顺序、解压时的字节数限制等，这些细节看似微小，却直接影响程序的运行结果。这让我认识到，在编写程序时，不仅要关注整体的算法逻辑，还要重视每一个细节的处理，养成严谨的编程习惯。

同时，**分步调试与模块化开发**是提高编程效率的关键。本次实验将整个程序拆分为多个模块，每个模块独立实现并测试，确保单个模块功能正常后再进行整体联调，大幅降低了调试的难度。在调试过程中，采用分步输出中间结果、设置断点查看变量的方式，能够快速定位问题所在，避免了在大量代码中盲目查找错误。

此外，本次实验也让我认识到自己的不足，例如对 C++ 标准库的使用不够熟练、对异常处理的考虑不够全面、对程序性能的优化能力不足等。在后续的学习中，我将加强 C++ 编程的练习，深入学习标准库的使用，完善程序的异常处理机制，同时学习程序性能优化的方法，进一步提升自己的编程能力。

总的来说，本次哈夫曼图片压缩实验是一次非常有意义的编程实践，不仅巩固了数据结构的理论知识，提升了 C++ 编程能力，还培养了问题解决能力和软件工程化思维，为后续的专业学习和工程实践打下了坚实的基础。

实验课程名称：　数据结构与算法综合实验

实验项目名称	图与景区信息管理系统			报告成绩	
实验者	王文硕	专业班级	软件卓越 2401	组别	
同组者				完成日期	2026.3.25

第一部分：实验分析与设计（可加页）

三、　实验目的和内容

1.实验目的

图结构的学习包括概念、定义、操作和编程。

通过“景区信息管理系统”的编程实践，学习图的遍历、Dijkstra 算法、最小生成树算法、Prim 算法和他们的编程应用。

开发和学习“景区信息管理系统”，达到如下目标：

(1)掌握图的定义和图的存储结构

(2)掌握图的创建方法

(3)掌握图的两种遍历方法

(4)理解迪杰斯特拉（Dijkstra）算法

(5)理解最小生成树的概念和普里姆（Prim）算法

(6)掌握文件操作

(7)使用 C++语言，利用图的数据结构，开发景区信息管理系统。

2.实验内容

1）业务背景

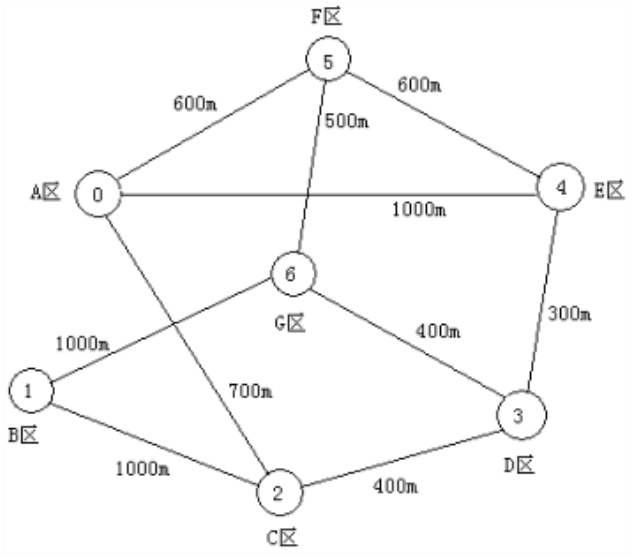
现有一个景区，景区里面有若干个景点，景点之间满足以下条件：

(1)某些景点之间铺设了道路（相邻）

(2)这些道路都可以双向行驶的（无向图）

(3)从任意一个景点出发都可以游览整个景区（连通图）

开发景区信息管理系统，对景区的信息进行管理。使用图的数据结构来保存景区景点信息，为用户提供创建图、查询景点信息、旅游景点导航、搜索最短路径、铺设电路规划等功能。



2)景点数据

景区的数据包含景点信息和景点之间的道路信息。分别由两个文本文件存储。Vex.txt 文件用来存储景点信息；Edge.txt 文件用来存储道路信息。

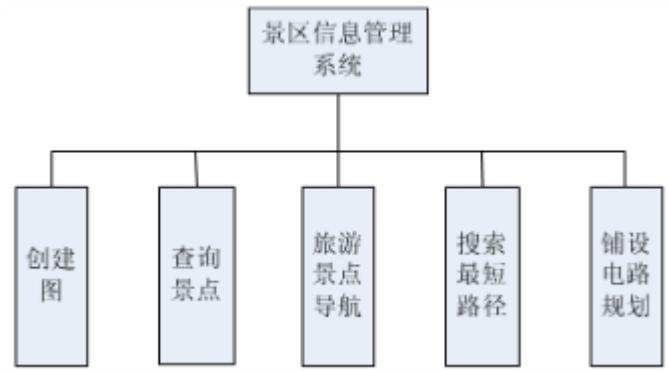
1.景点信息：景点编号、名字和介绍

编号	名字	介绍
0	A 区	…
1	B 区	…
2	C 区	…
3	D 区	…
4	E 区	…
5	F 区	
6	G 区	…

2.道路信息：景点 1，景点 2，两个景点之间的距离

景点 1	景点 2	距离（m）
A	C	700
A	E	1000
A	F	600
B	C	1000
B	G	1000
C	D	400
D	E	300
D	G	400
E	F	600
F	G	500

3）系统功能



1.创建图

输入：从 Vex.txt 文件中读取景点信息，从 Edge.txt 文件中读取道路信息。

处理：根据读取的景点信息创建景区景点图。

输出：

创建成功，一次输出：

（1）顶点数目

（2）顶点编号

（3）顶点名字

（4）边两端的顶点编号

（5）边的权值

创建失败，输出失败的提示信息。

2.查询景点

输入：想要查询的景点的编号

处理：根据输入的景点编号，查询该景点及相邻景点的信息。

输出：

（1）景点名字

（2）景点介绍

（3）相邻景点的名字

（4）到达相邻景区的路径长度

3.旅游景点导航

输入：起始景点的编号

处理：使用深度优先搜索（DFS）算法,查询以该景点为起点，无回路游览整个景区的路线。

输出：所有符合要求的导航路线。

4.搜索最短路径

输入：

（1）起始景点的编号

（2）终点的编号

处理：使用迪杰斯特拉算法，求得从起始景点到终点之间的最短路径，计算路径总长度。

输出：

（1）最短路线

（2）路径总长度

5.铺设电路规划

输入：景区的所有景点信息和道路信息

处理：根据景区景点图使用普里姆（Prim）算法构造最小生成树，设计出一套铺设线路最短，但能满足每个景点都能通电的方案。

输出：

（1）需要铺设电路的道路

（2）每条道路铺设电路的长度

（3）铺设电路的总长度

四、 分析与设计

1.数据结构的设计

本次实验采用邻接矩阵存储无向连通图，结合结构体和类实现数据的封装，核心数据结构设计如下：

（1）顶点结构体（Vex）

存储单个景点的信息，包含编号、名字、介绍三个字段，适配 Vex.txt 的读取格式。

```
struct Vex {
    int num;        // 景点编号
    char name[20];  // 景点名字
    char desc[1024];// 景点介绍
};
```

（2）边结构体（Edge）

存储单条道路的信息，包含两个顶点编号、边的权值（距离），适配 Edge.txt 的读取格式。

```
struct Edge {
    int vex1;       // 边的第一个顶点编号
    int vex2;       // 边的第二个顶点编号
    int weight;     // 边的权值（道路距离，单位 m）
};
```

（3）路径节点结构体（PathNode）

用于旅游导航功能，采用链表存储 DFS 遍历得到的所有游览路线，支持动态添加路线。

```
typedef struct PathNode {
    int vexs[MAX_VERTEX_NUM]; // 存储一条路线的所有景点编号
    struct PathNode* next;    // 指向下一条路线的指针
} PathNode, *PathList;
```

（4）图的类（CGraph）

采用面向对象思想封装图的所有操作，私有成员为邻接矩阵、顶点数组、顶点数目，公有成员为图的初始化、顶点 / 边插入、遍历、最短路径、最小生成树等核心方法，实现算法与数据的解耦。

2.核心算法设计

（1）改进的深度优先搜索（DFS）算法（旅游导航）

在基础 DFS 算法上增加回溯机制，实现多路线生成：

1. 定义 bVisited 数组标记顶点访问状态，aPath 数组记录当前遍历路径，nIndex 记录遍历深度；
2. 从起始顶点出发，递归访问未标记的邻接顶点，标记为已访问并加入路径；
3. 若所有顶点均被访问，将当前路径保存到链表中；
4. 递归返回后执行回溯：将顶点标记为未访问，实现其他路线的探索；
5. 最终遍历路径链表，输出所有无回路的全景区游览路线。

（2）迪杰斯特拉（Dijkstra）算法（最短路径搜索）

采用贪心策略求解单源最短路径，核心步骤：

1. 定义 dist 数组存储源点到各顶点的最短距离，preVex 数组记录各顶点的前驱顶点，bVisited 数组标记顶点是否加入最短路径集；
2. 初始化：源点到自身距离为 0，到邻接顶点距离为边的权值，到非邻接顶点距离为无穷大；
3. 循环 n-1 次（n 为顶点数），每次选择距离源点最近的未访问顶点 u，标记为已访问；
4. 以 u 为中间点，更新源点到其他未访问顶点的最短距离，若 dist[u]+权值 < dist[v] 则更新并记录前驱；
5. 遍历结束后，通过 preVex 数组回溯得到源点到终点的最短路径，dist[终点]即为最短距离。

（3）普里姆（Prim）算法（电路铺设规划）

以顶点 0（A 区）为起始点构造最小生成树，满足连通所有顶点且总权值最小，核心步骤：

1. 定义 dist 数组存储非生成树顶点到生成树的最小距离，preVex 数组记录非生成树顶点的前驱，inTree 数组标记顶点是否加入生成树；
2. 初始化：起始顶点加入生成树，dist 数组为起始顶点到各顶点的边权值；
3. 循环 n-1 次，每次选择非生成树中 dist 最小的顶点 u，将边(preVex[u], u)加入生成树；
4. 以 u 为中间点，更新非生成树顶点到生成树的最小距离；
5. 最终生成树的边即为电路铺设的道路，所有边的权值和为总铺设长度。

三、主要仪器设备及耗材

3. 安装了 Windows 11 的 PC 机 1 台
4. PC 机系统上安装了 Microsoft Visual Studio 2022 开发环境

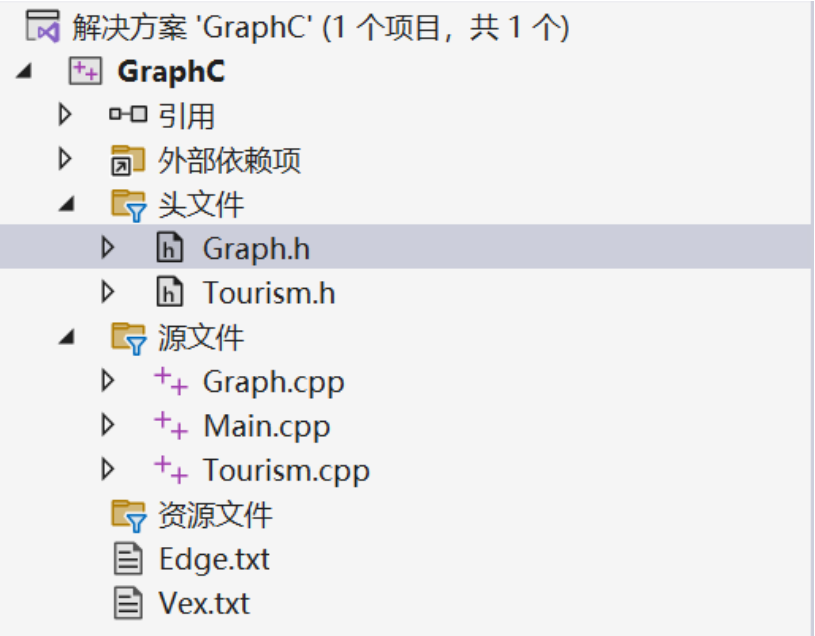
第二部分：实验过程和结果（可加页）

三、 实现说明

本次实验采用迭代开发模式，基于 Microsoft Visual Studio 创建控制台工程 GraphCPro，工程结构分为头文件和源文件，各文件的功能分工如下：
1.头文件 Graph.h 定义顶点、边、路径链表结构体，声明 CGraph 类的所有成员函数
2.头文件 Tourism.h 声明全局图对象，定义景区管理的业务函数（创建图、查询景点、导航等）
3.源文件 Graph.cpp 实现 CGraph 类的所有方法，包含图的操作、DFS、Dijkstra、Prim 核心算法
4.源文件 Tourism.cpp 实现景区业务逻辑，完成文件读取、界面交互、算法调用、结果输出
5.源文件 Main.cpp 程序入口，实现控制台菜单交互，接收用户输入并调用对应业务函数

实验的核心实现流程为：工程搭建→数据结构定义→图的创建（文件读取）→基础功能实现（查询）→算法功能实现（DFS/Dijkstra/Prim）→界面交互→调试运行，所有功能均基于 C++ 语言实现，严格遵循模块化和面向对象设计思想。

四、 源代码



五、 调试说明（调试手段、过程及结果分析）

（一） 调试手段与过程

本次实验采用**分步调试**和**断点调试**相结合的方式，解决开发过程中的问题，核心调试过程与结

果分析如下：

1. **文件读取调试**：初始读取 Vex.txt 时出现景点介绍读取为空的问题，原因是未处理换行符，添加 fin.ignore()后解决，实现文件数据的正确读取；
2. **邻接矩阵调试**：无向图的边需双向赋值，初始仅赋值 m_aAdjMatrix[v1][v2]，导致邻接顶点查询失败，补充 m_aAdjMatrix[v2][v1]后问题解决；
3. **DFS 算法调试**：初始未实现回溯机制，仅生成一条路线，添加 bVisited[w] = false 后实现多路线生成，满足旅游导航的功能要求；
4. **Dijkstra 算法调试**：无穷大值定义过小导致整数溢出，将 INF 改为 0x3f3f3f3f 后解决，保证最短距离计算的正确性；
5. **Prim 算法调试**：初始误将 dist 数组定义为源点到各顶点的距离，修正为**非生成树到生成树的最小距离**后，成功生成最小生成树；
6. **界面交互调试**：对用户输入的景点编号增加合法性校验，避免数组下标越界，提升程序的健壮性。

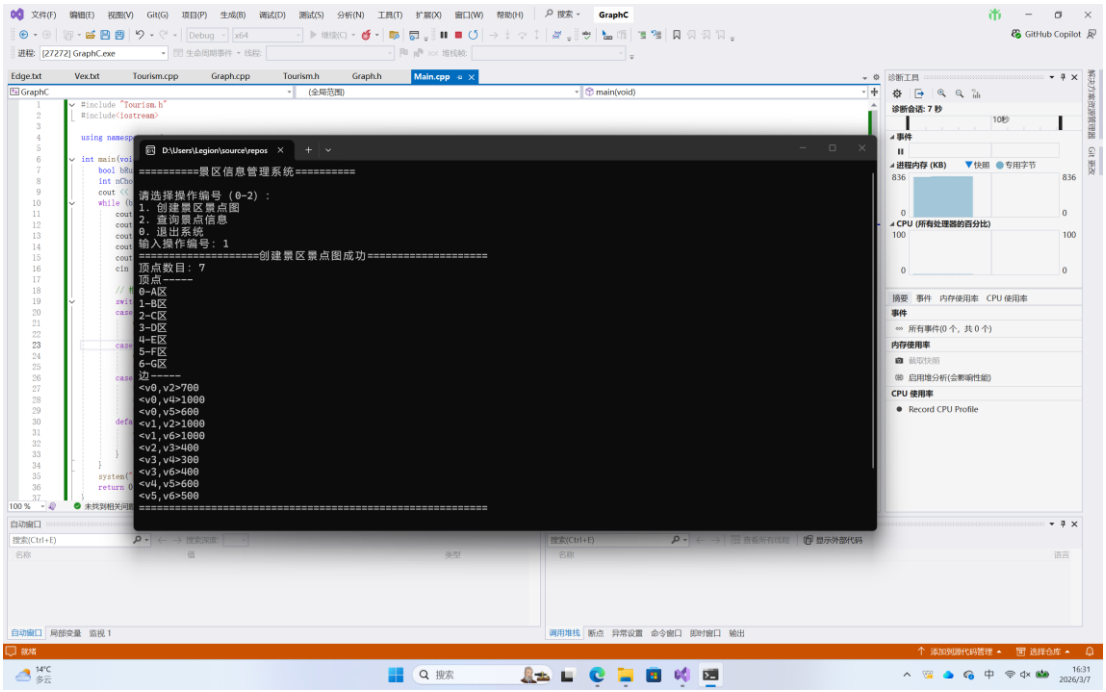
所有调试完成后，程序可正常编译运行，无语法错误和逻辑错误，各功能模块均能实现预期效果。

（二） 实验结果分析

实验运行基于 Microsoft Visual Studio 2022，按**创建图→功能调用**的顺序操作，核心功能的运行结果如下：

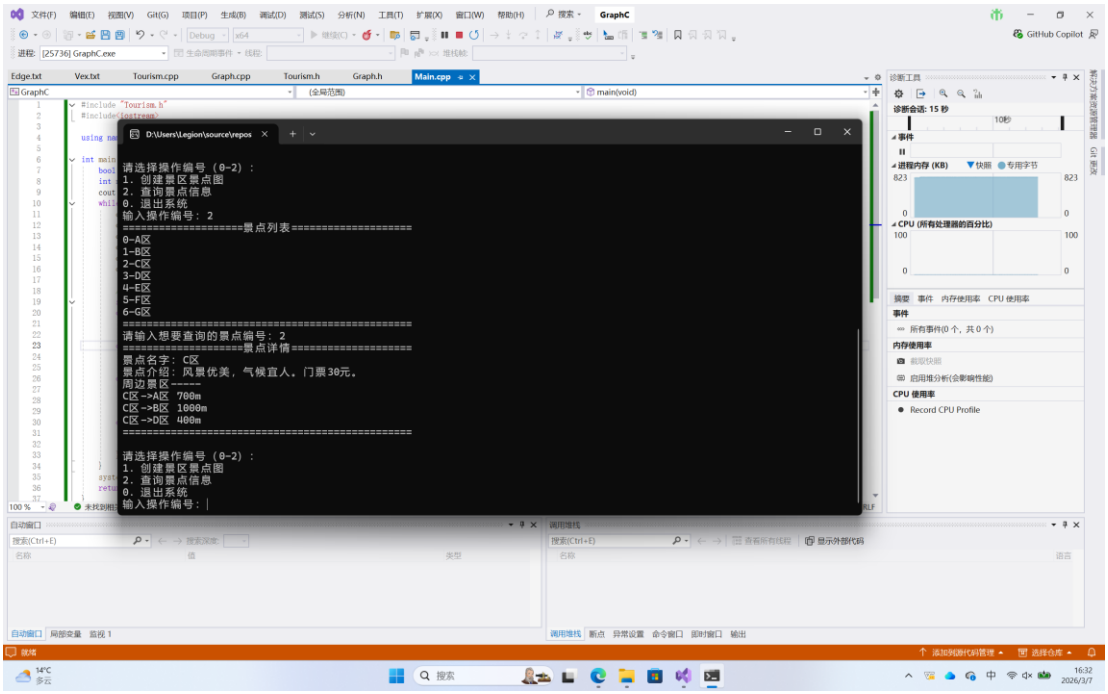
1. 创建景区景点图

输入操作编号 1，程序读取 Vex.txt 和 Edge.txt，创建邻接矩阵图并输出结果，关键输出如下：



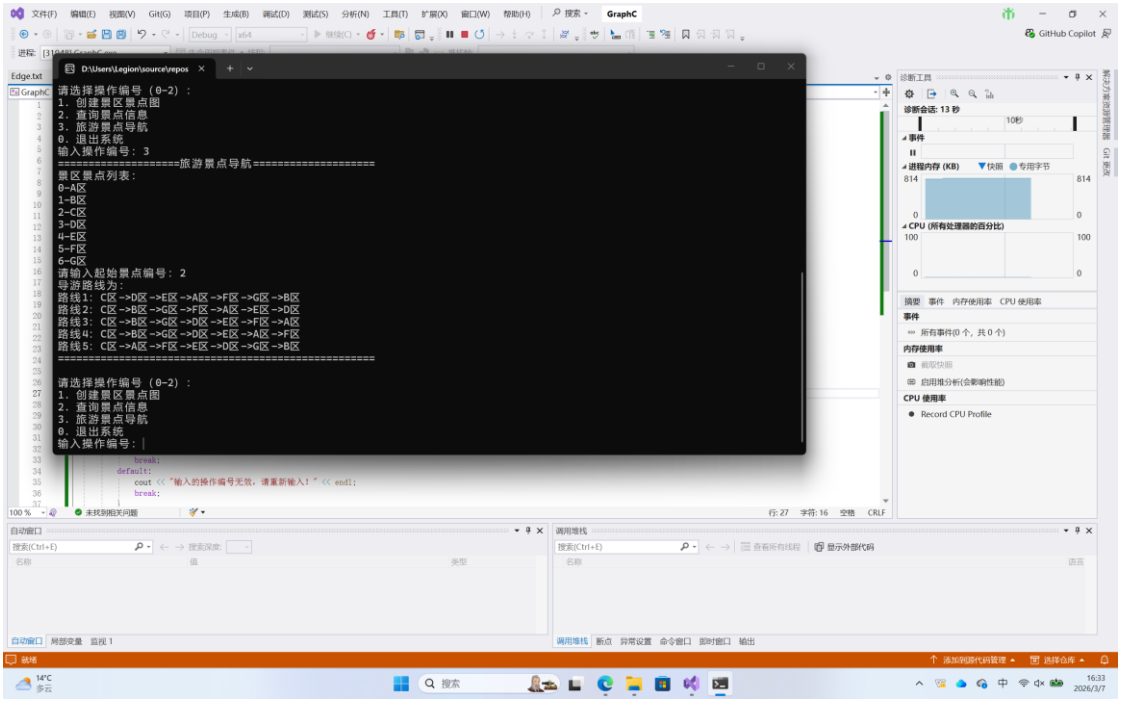
2. 查询景点信息

输入操作编号 2，选择查询景点 2（C 区），输出结果如下：



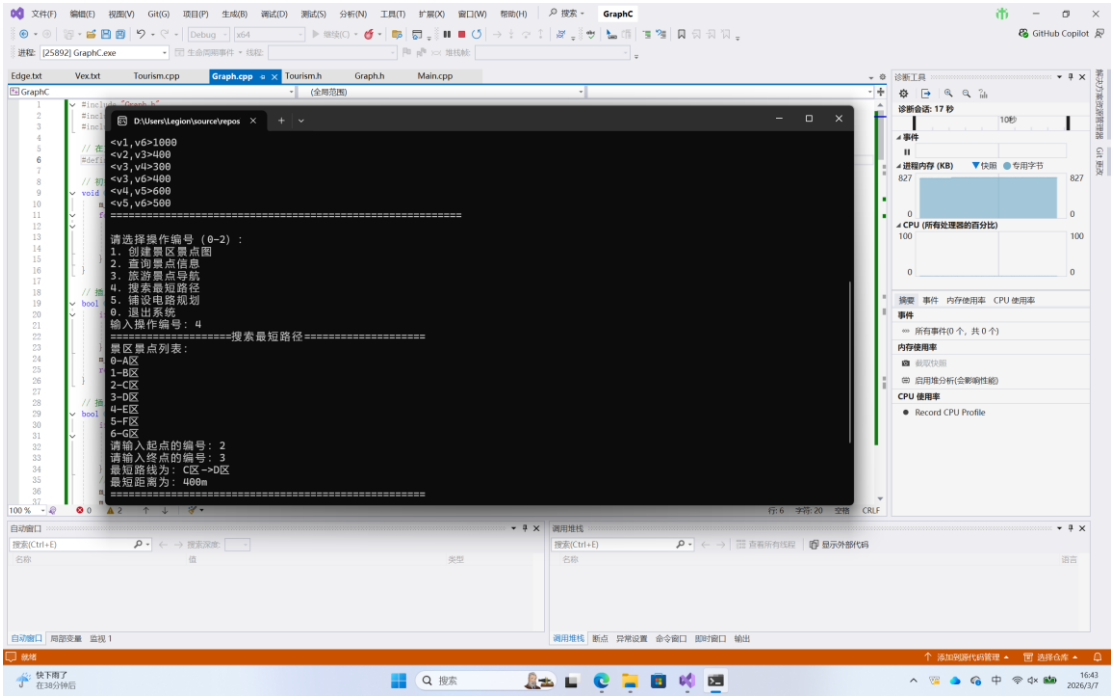
3. 旅游景点导航

输入操作编号 3，选择起始景点 2（C 区），程序输出多条无回路游览路线，关键输出如下：



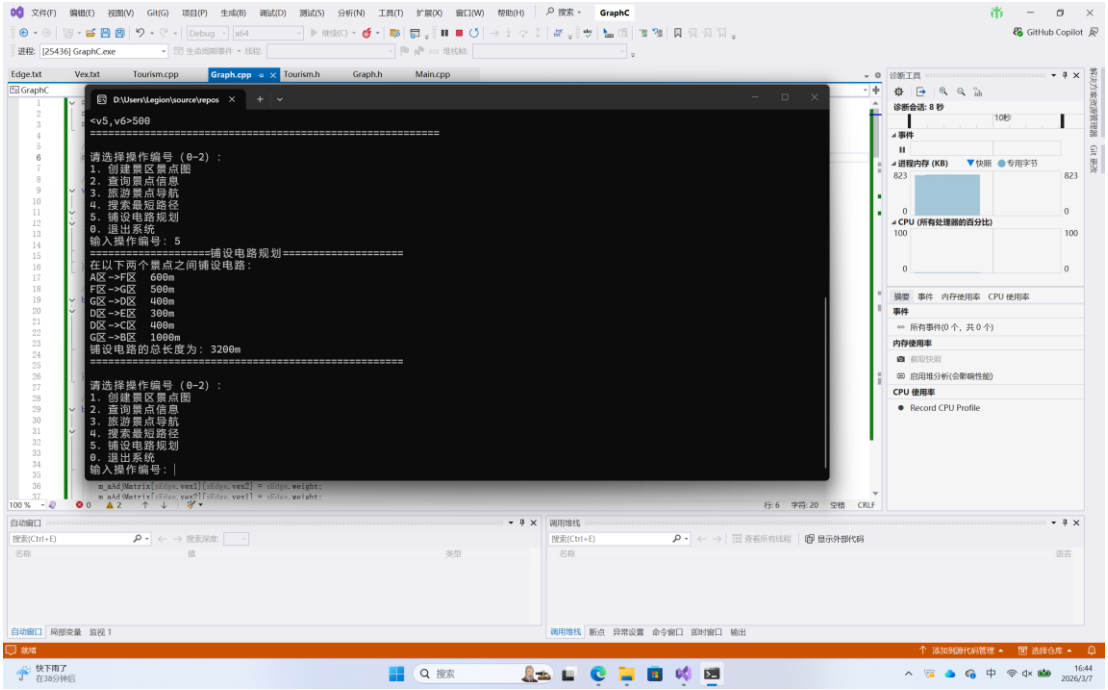
4. 搜索最短路径

输入操作编号 4，选择起点 2（C 区）、终点 3（D 区），输出结果与实验预期一致：



5. 铺设电路规划

输入操作编号 5，程序通过 Prim 算法生成最小生成树，输出电路铺设方案，总长度与实验预期一致：



第三部分：实验小结、收获与体会

一、实验小结

本次实验成功完成了**景区信息管理系统**的开发，基于 C++ 语言和邻接矩阵存储结构，实现了创建图、查询景点、旅游导航、最短路径搜索、电路铺设规划五大核心功能。实验过程中严格遵循迭代开发模式，从基础的图创建到复杂的图算法实现，逐步完成功能模块的开发与整合，最终程序可正常编译运行，各功能的运行结果均与实验预期一致，成功将图的相关算法应用到实际的景区信息管理业务场景中。

本次实验的核心重难点为**改进的 DFS 算法回溯机制、Dijkstra 算法的贪心策略与路径回溯、Prim 算法的最小生成树构造**，通过分步调试和逻辑梳理，成功解决了开发过程中的文件读取、算法逻辑、界面交互等问题，实现了算法的正确落地。同时，实验采用模块化和面向对象的设计思想，将图的底层算法与景区业务逻辑解耦，提升了程序的可读性、可维护性和可扩展性。

二、实验收获

1. **知识层面**：深入掌握了图的邻接矩阵存储结构，理解了无向连通图的创建方法；熟练掌握了深度优先搜索（DFS）的遍历原理，并能根据业务需求进行改进；理解了 Dijkstra 算法的贪心思想和 Prim 算法的最小生成树构造原理，掌握了两大算法的具体实现步骤；巩固了 C++ 文件操作、结构体、类、链表等基础知识点。
2. **能力层面**：提升了 C++ 编程能力和面向对象的程序设计能力，学会了模块化的代码组织方式；培养了算法设计与实现能力，学会了将抽象的算法思想转化为具体的代码；提升了程序调试能力，掌握了分步调试、断点调试等方法，能够快速定位并解决程序中的语法和逻辑错误；培养了问题分析与解决能力，能结合业务场景分析算法的应用场景，选择合适的算法解决实际问题。
3. **工程层面**：了解了迭代开发的模式，学会了从基础功能到复杂功能的逐步开发；掌握了控制台程序的界面交互设计，学会了根据用户输入实现功能的调用；理解了数据持久化的基本思想，学会了从文本文件中读取数据并应用到程序中。

三、实验体会

本次实验让我深刻体会到了**数据结构与算法在程序设计中的核心作用**，景区信息管理系统的所有功能均基于图的相关算法实现，合适的算法能高效解决实际业务问题：例如 DFS 算法解决了旅游导航的多路线生成问题，Dijkstra 算法高效求解了两点间的最短路径，Prim 算法实现了电路铺设的最优规划。同时，我也体会到了**算法与业务场景结合的重要性**，抽象的算法需要根据实际业务需求进行调整和改进，例如基础的 DFS 算法仅能生成一条遍历路线，通过增加回溯机制后，才能满足旅游导航的多路线需求。

在实验开发过程中，我也认识到了**编程规范和调试能力的重要性**。良好的编程规范能提升代码的可读性和可维护性，而高效的调试能力能快速定位程序中的问题，提升开发效率。例如在文件读取阶段，因未处理换行符导致数据读取错误，通过调试快速定位问题后，仅需添加一行代码即可解决。同时，面向对象的设计思想让程序的结构更清晰，算法与业务逻辑的解耦让后续的功能扩展更便捷。

此外，本次实验也让我体会到了**循序渐进、分步实现**的开发思路的重要性。从工程搭建到数据结构定义，从图的创建到基础功能实现，再到复杂算法的落地，每一步都为后续的开发奠定基础，避免了因一步到位而导致的逻辑混乱。在今后的编程学习中，我将继续加强数据结构与算法的学习，提升算法设计与实现能力，同时注重编程规范和调试能力的培养，学会用工程化的思想进行程序设计，将所学知识应用到更多的实际业务场景中。

四、实验不足与改进方向

1. **实验不足**：本次实验采用邻接矩阵存储图，当景点数量较多时，邻接矩阵会存在空间浪费的问题；旅游导航功能输出的路线较多，未对路线进行优化和排序；程序仅实现了文

件的读操作，未实现写操作，无法保存用户的操作数据；界面为控制台交互，较为简陋，用户体验一般。

2. **改进方向**：后续可采用**邻接表**存储图，减少空间浪费，提升程序对大规模数据的适配性；对旅游导航的路线进行优化，例如按路径长度排序，为用户提供更优的游览路线；增加文件的写操作，实现用户操作数据的持久化保存；基于 MFC 或 Qt 开发图形化界面，提升用户体验；增加更多功能模块，例如景点添加 / 删除、道路信息修改等，完善景区信息管理系统的功能。

实验课程名称：　数据结构与算法综合实验

实验项目名称	线性结构与连连看游戏			报告成绩	
实验者	王文硕	专业班级	软件卓越 2401	组别	
同组者				完成日期	2026. 4. 25

第一部分：实验分析与设计（可加页）

五、　实验目的和具体内容

1. 实验目的

通过连连看项目，达到如下目标：

(1) 了解业务背景，调研与连连看同类型游戏，了解连连看游戏的功能和规则等。

(2) 掌握 C++开发工具和集成开发环境（Microsoft Visual Studio 2022）

(3) 掌握 C++面向对象的编程思想和 C++的基础编程。

(4) 了解 MFC 基本框架，包括 MFC Dialog 应用程序和 GDI 编程。

(5) 了解线性结构，重点掌握数组和栈操作，数组遍历、消子和胜负判断等算法。

(6) 了解项目开发流程，了解系统需求分析和设计，应用迭代开发进行项目开发。

(7) 养成良好的编码习惯和培养软件工程化思维，综合应用“C++编程、MFC Diaolog、算法、线性结构”等知识，开发“连连看游戏”桌面应用程序，达到掌握和应用线性结构核心知识的目的。

2. 实验内容

实现基本功能：开始游戏、暂停游戏、消子、判断胜负、提示、重排、计时等。

（1）主界面：设计“欢乐连连看”项目的主界面，在主界面上添加一个背景图片，并在适当的地方添加“基本模式”、“休闲模式”、“关卡模式”、“帮助”、“设置”、“排行榜”按钮。

（2）开始游戏：当玩家在主界面选择“基本模式”时，出现基本游戏界面，并隐藏主界面，玩家点击“开始游戏”按钮，生成游戏地图。

（3）消子：对玩家选中的两张图片进行判断，判断是否符合消除规则。符合一条直线连通、两条直线连通、三条直线连通这三种情况之一就可以消除。如果可以消除，从游戏地图中提示连接路径，然后消除这两张图片。如果不能消除，则保持原来的游戏地图。

消子规则

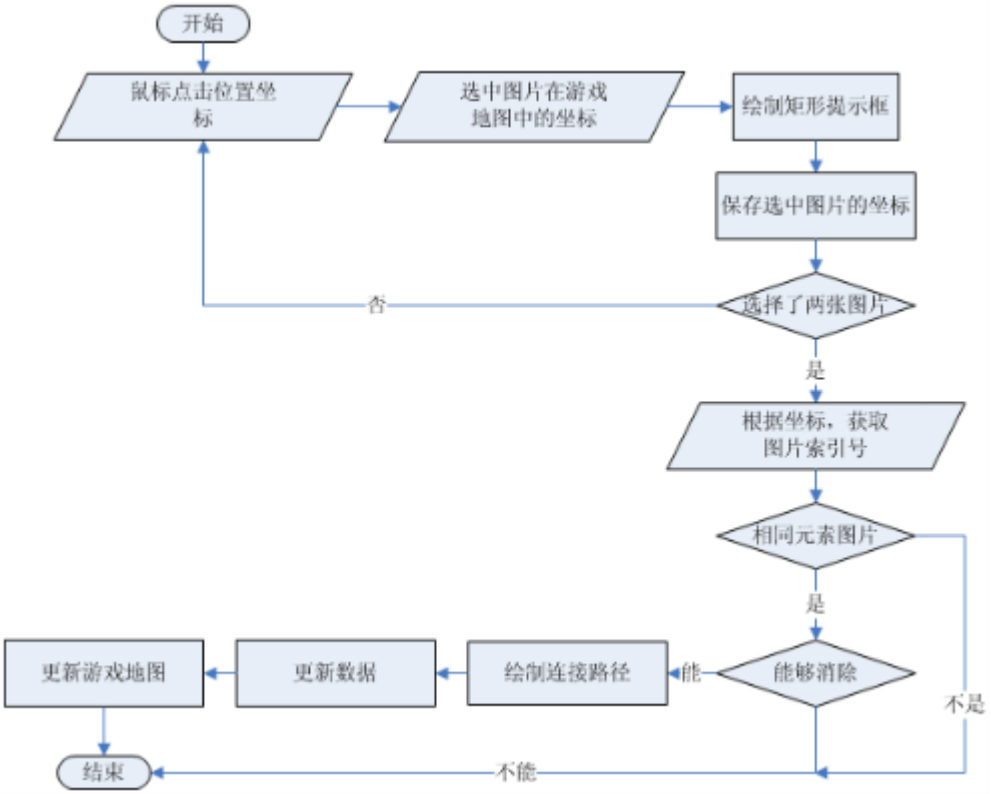
（4）判断胜负：在基本模式下如果将游戏地图中的所有的图片都消除，则提示玩家获胜，并且可以重新开始新游戏。

（5）提示：可以提示界面上能够消除的一对图片。

（6）重排：根据随机数，重新排列游戏地图上的图片。

（7）计时：设定一定的时间来辅助游戏是否结束。

（8）暂停游戏：游戏过程中可以暂停计时，并且将游戏地图遮盖，按钮显示为继续游戏。选择继续游戏，计时继续。



六、 分析与设计

欢乐连连看项目采用 MFC 框架，软件采用三层结构。使用二维数组来保存游戏地图中的数据，基本实现了连连看的核心功能。

1.数据结构的设计

（1）Vertex 顶点结构体：用于记录游戏地图中图案 / 空白点的行、列坐标及图案编号，是消子判断、路径记录的基础，定义于 global.h：

```
typedef struct tagVertex{
    int row;    // 行号
    int col;    // 列号
    int info;   // 图案编号，BLANK(0)表示空白
}Vertex;
```

（2）二维动态数组：作为游戏地图的存储结构，int** m_pGameMap，其中 m_pGameMap[i][j]

表示第 i 行第 j 列的图案编号，0 为空白，非 0 为对应图案编号。

（3）路径顶点数组：Vertex m_avPath[4]，用于记录连通路径的顶点，最多存储 4 个点（起点 + 2 个拐点 + 终点），满足三条直线连通的路径记录需求。

2.界面设计

3.类设计

1) 数据成员

类名	所属层级	核心职责	关键数据成员	核心成员函数
CLLKApp	入口	程序初始化、消息循环	-	InitInstance()（程序初始化）
CLLKDlg	界面层	游戏主界面（模式选择）	CDC m_dcMem、CBitmap m_bmpBG	InitBackground()、OnBnClickedBtnBasic()
CGameDlg	界面层	游戏核心界面（图案交互）	CGameControl m_gameCtr、bool m_bPlaying、CRect m_rtGameRect	OnLButtonUp()、DrawTipFrame()、DrawTipLine()、UpdateMap()
CHelpDlg	界面层	帮助对话框	-	OnInitDialog()（对话框初始化）
CGameControl	控制层	游戏状态管理、层间协调	int** m_pGameMap、Vertex m_svSelFst/sec、CGameLogic m_gameLogic	StartGame()、Link()、FindTip()、ShuffleMap()、IsWin()
CGameLogic	逻辑层	核心算法实现	Vertex m_avPath[4]、int m_nVexNum	InitMap()、IsLink()、LinkInRow/Col()、OneCornerLink()、TwoCornerLink()、IsBlank()

2) 成员函数

CGameLogic::IsLink：消子连通判断核心函数，依次判断**一条直线**、**两条直线**、**三条直线**连通，返回是否可消除并记录路径；

CGameLogic::InitMap：动态生成游戏地图，完成二维数组分配、图案填充、随机打乱，适配经典 / 休闲模式的不同尺寸；

CGameControl::Link：控制层消子入口，判断选中图案是否为同一种、调用逻辑层连通判断、消除图案并更新地图；

CGameDlg::OnLButtonUp：鼠标左键抬起事件响应，计算点击坐标对应的地图行列、处理图案选择、调用消子功能并判断胜负；

CGameDlg::DrawTipFrame/DrawTipLine：绘制选中图案的红色提示框和消除的绿色连通路径，提升玩家体验；

CGameControl::FindTip：提示功能核心，遍历所有图案对查找可消除组合，调用逻辑层连通判断并返回路径；

CGameControl::ShuffleMap：重排功能核心，收集非空白图案并随机打乱，重新填充到地图中。

三、主要仪器设备及耗材

1. 安装了 Windows 11 的 PC 机 1 台
2. PC 机系统上安装了 Microsoft Visual Studio 2022 开发环境
3. 开发语言：C++（面向对象）+ MFC GDI 编程；
4. 资源文件：背景位图（IDB_MAIN_BG、IDB_GAME_BG）、元素位图（IDB_ELEMENT）、掩码位图（IDB_MASK）、图标文件等。

第二部分：实验过程和结果（可加页）

六、 实现说明

1、项目工程结构

本项目基于 MFC Dialog 创建工程，工程名称为 LLK，解决方案名称为 Lianliankan，工程结构按**头文件**、**源文件**、**资源文件**分类，核心文件如下：

类型	核心文件	功能说明
头文件	LLK.h、LLKDlg.h、GameDlg.h、HelpDlg.h、GameControl.h、GameLogic.h、global.h、resource.h	定义应用程序、对话框、控制层、逻辑层类，声明全局常量 / 结构体
源文件	LLK.cpp、LLKDlg.cpp、GameDlg.cpp、HelpDlg.cpp、GameControl.cpp、GameLogic.cpp、pch.cpp	实现头文件中声明的类的成员函数，核心算法与界面逻辑实现
资源文件	LLK.rc、res 文件夹（位图、图标）	管理游戏的位图、图标、对话框、按钮等资源

2、核心功能实现步骤

本项目按迭代开发分为 6 个核心步骤，每一步完成一个核心功能，步骤间层层递进：

步骤 1：创建 MFC 工程与主界面设计

打开 VS2015,创建空白解决方案 Lianliankan,添加 MFC Dialog 工程 LLK,设置对话框为 Dialog based，添加最小化按钮；

修改主对话框 CLLKDlg 的标题为 “欢乐连连看”，设置窗口大小为 800*600 并居中，替换程序图标；

导入主界面背景位图（IDB_MAIN_BG），通过 GDI 编程实现背景绘制（InitBackground()+OnPaint()）；

添加基本模式、休闲模式、帮助按钮，绑定按钮点击事件，实现模式选择与界面跳转。

步骤 2：游戏界面创建与地图初始化

添加游戏对话框资源 IDD_GAME_DIALOG，创建游戏界面对话框类 CGameDlg；

实现主界面到游戏界面的跳转（CLLKDlg::OnBnClickedBtnBasic()），隐藏主界面并显示游戏界面；

导入游戏背景、元素、掩码位图，通过透明贴图原理实现图案的无背景绘制；

逻辑层实现 CGameLogic::InitMap()，动态分配二维数组，填充成对图案并随机打乱；控制层调用该函数完成游戏地图初始化（CGameControl::StartGame()）；

界面层实现 UpdateMap()，遍历二维数组并绘制游戏地图，完成 “开始游戏” 功能。

步骤 3：消子功能实现（核心）

鼠标事件响应：给 CGameDlg 添加 WM_LBUTTONUP 消息响应函数 OnLButtonUp()，计算鼠标点击坐标对应的地图行列，判断是否点击有效图案；

图案选择：通过 bool m_bFirstSel 标识首次 / 二次选择，首次选择绘制红色提示框，二次选择记录两个图案的 Vertex 坐标；

消子判断：

控制层 CGameControl::Link()判断两个图案是否为同一种、是否为同一个点；

逻辑层 CGameLogic::IsLink()依次调用 LinkInRow/Col()（一条直线）、OneCornerLink()（两条直线）、TwoCornerLink()（三条直线）完成连通判断；

连通判断通过数组遍历实现路径检测，确保路径中无其他图案，同时通过 m_avPath[4]记录连通路径；

图案消除与路径绘制：若连通判断成功，将地图中两个图案设为空白（BLANK），调用 DrawTipLine()绘制绿色连通路径，延迟 180ms 后更新地图。

步骤 4：胜负判断功能实现

逻辑层添加 CGameLogic::IsBlank()，遍历游戏地图二维数组，判断是否所有元素均为空白；

控制层添加 CGameControl::IsWin()，调用逻辑层的 IsBlank()函数，返回游戏是否胜利；

界面层在消子成功后（OnLButtonUp()）调用 IsWin()，若胜利则弹出 “恭喜你，游戏胜利!” 提示框，将 m_bPlaying 设为 false，启用 “开始游戏” 按钮。

步骤 5：提示功能实现

给游戏界面添加提示按钮，绑定点击事件 CGameDlg::OnBnClickedBtnTip()；

控制层实现 CGameControl::FindTip()，双重遍历游戏地图的所有非空白图案对，调用 IsLink()查找可消除组合；

若找到可消除组合，界面层调用 DrawTipFrame()和 DrawTipLine()绘制红色提示框和绿色路径；若无则弹出 “当前无可用消除路径，请点击重排!” 提示框。

步骤 6：重排功能实现

给游戏界面添加重排按钮，绑定点击事件 CGameDlg::OnBnClickedBtnReset()；

控制层实现 CGameControl::ShuffleMap()，收集地图中所有非空白图案的 Vertex 信息，通过

random_shuffle 随机打乱；
将打乱后的图案重新填充到原地图的非空白位置，调用 UpdateMap()更新游戏界面，生成新的可消除可能。

七、 源代码

核心代码片段:

1. 消子连通判断核心（CGameLogic::IsLink）

```
bool CGameLogic::IsLink(int** pGameMap, Vertex v1, Vertex v2, Vertex avPath[4], int& nVexNum, int rows, int cols)
{
    ClearPath();
    PushVertex(v1); // 起点入栈
    // 判断是否为同一个点/图案是否一致
    if (v1.row == v2.row && v1.col == v2.col) return false;
    if (pGameMap[v1.row][v1.col] != pGameMap[v2.row][v2.col]) return false;
    // 1. 一条直线连通（行/列）
    if (LinkInRow(pGameMap, v1, v2, GAME_COLS) || LinkInCol(pGameMap, v1, v2, GAME_ROWS))
    {
        PushVertex(v2);
        nVexNum = m_nVexNum;
        memcpy(avPath, m_avPath, sizeof(Vertex) * m_nVexNum);
        return true;
    }
    // 2. 两条直线连通（一个拐点）
    if (OneCornerLink(pGameMap, v1, v2))
    {
        PushVertex(v2);
        nVexNum = m_nVexNum;
        memcpy(avPath, m_avPath, sizeof(Vertex) * m_nVexNum);
        return true;
    }
    // 3. 三条直线连通（两个拐点）
    if (TwoCornerLink(pGameMap, v1, v2))
    {
        PushVertex(v2);
        nVexNum = m_nVexNum;
        memcpy(avPath, m_avPath, sizeof(Vertex) * m_nVexNum);
        return true;
    }
    ClearPath();
    return false;
}
```

2. 鼠标选择与消子处理（CGameDlg::OnLButtonUp）

```
void CGameDlg::OnLButtonUp(UINT nFlags, CPoint point){
    if (!m_bPlaying || !m_rtGameRect.PtInRect(point)) // 游戏未开始/点击地图外，不响应
    {
        CDialogEx::OnLButtonUp(nFlags, point);
        return; }
    // 计算点击的行和列
    int nRow = (point.y - GAME_TOP_Y) / ELEM_SIZE;
    int nCol = (point.x - GAME_TOP_X) / ELEM_SIZE;
    if (m_gameCtr.GetElement(nRow, nCol) == BLANK) // 点击空白，不响应
    {
        CDialogEx::OnLButtonUp(nFlags, point);
        return;
    }
    // 首次选择
    if (m_bFirstSel){
        m_gameCtr.SetFirstPoint(nRow, nCol);
        DrawTipFrame(nRow, nCol); // 绘制红色提示框
        m_bFirstSel = false;
    }
    // 二次选择
    else{
        m_gameCtr.SetSecPoint(nRow, nCol);
        Vertex avPath[4] = { 0 };
        int nVexNum = 0;
        bool bSuc = m_gameCtr.Link(avPath, nVexNum); // 消子判断
        if (bSuc){
            DrawTipLine(avPath, nVexNum); // 绘制绿色连通路径
            UpdateMap(); // 更新地图
            // 判断胜负
            if (m_gameCtr.IsWin()){
                MessageBox(_T("恭喜你，游戏胜利！"), _T("提示"), MB_OK |
MB_ICONINFORMATION);
                m_bPlaying = false;
                GetDlgItem(IDC_BTN_START)->EnableWindow(TRUE); // 启用开始按钮
            }
        }else{
            UpdateMap(); // 消子失败，重绘地图清除提示框
        }
        m_bFirstSel = true; // 重置选择状态
    }
    CDialogEx::OnLButtonUp(nFlags, point);
}
```

八、 调试说明（调试手段、过程及结果分析）

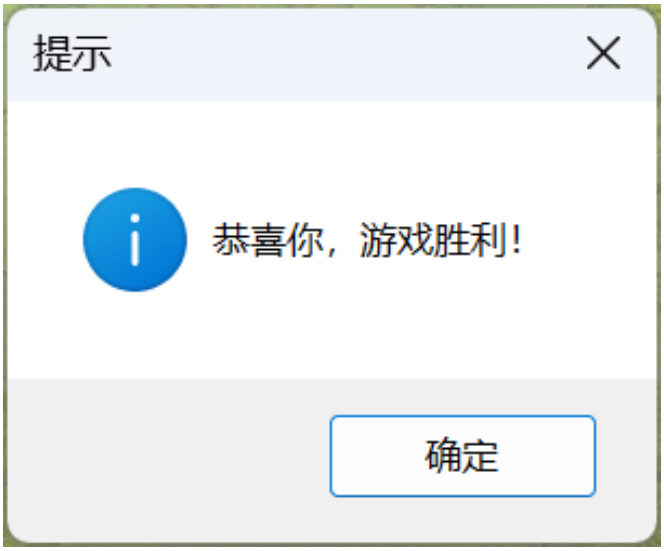
（一）实验运行结果

1. **主界面**：启动程序后显示 800*600 的主界面，背景图片正常显示，“基本模式”“休闲模式”“帮助” 按钮可正常点击，窗口居中且大小固定；

2. **游戏界面**：选择模式后跳转到游戏界面，点击 “开始游戏” 按钮，动态生成对应尺寸的游戏地图，图案无白色背景、均匀分布且随机打乱；

3. **图案选择**：鼠标点击图案，图案周围绘制红色边框，二次点击其他图案时，自动判断是否为同一种；

4. **消子功能**：
   - 符合连通规则时，绘制绿色的连通路径（直线 / 拐点），延迟 180ms 后图案消失，地图对应位置变为空白；
   - 不符合规则时，重绘地图清除红色提示框，地图无变化；
5. **胜负判断**：当地图中所有图案均被消除时，弹出胜利提示框，“开始游戏” 按钮从禁用变为启用，点击可重新生成地图；



6. **提示功能**：点击 “提示” 按钮，若有可消除组合，对应图案绘制红色边框并显示绿色路径；若无则弹出提示弹窗；

7. **重排功能**：点击 “重排” 按钮，地图中剩余图案随机打乱，界面实时更新，生成新的可消除可能；
8. **资源释放**：关闭窗口时，位图、DC、二维数组等资源均被正常释放，无内存泄漏。

（二）调试过程与问题解决

本次实验开发过程中遇到多个典型问题，通过**断点调试**、**单步执行**、**日志输出**等方式解决，核心问题及解决方法如下：

1. **问题 1**：图案绘制出现白色背景，与游戏背景不协调。
   - 解决：使用**掩码位图 + 位运算**，先将掩码位图与背景做 SRCPAINT，再将元素位图与结果做 SRCAND，消除白色背景实现透明贴图。
2. **问题 2**：消子判断时，同一行 / 列的图案中间有空白仍无法消除。
   - 解决：修改 LinkInRow/Col()的遍历逻辑，将循环条件从 i < nCol2 改为 i <= nCol2，允许路径中包含空白，仅排除非空白图案。
3. **问题 3**：地图外的拐点无法识别，导致部分边缘图案无法消除。
   - 解决：修改 OneCornerLink()和 TwoCornerLink()的拐点判断逻辑，若拐点在地图外，直接判定为合法（无需判断是否为空白）。
4. **问题 4**：重排功能后，部分图案消失或重复。
   - 解决：优化 ShuffleMap()的图案收集逻辑，确保仅收集非空白图案，打乱后仅填充到原地图的非空白位置，保持空白位置不变。

5. **问题 5**：游戏胜利后，“开始游戏” 按钮未启用。
    - 解决：在胜负判断的 if 语句中，添加 GetDlgItem(IDC_BTN_START)->EnableWindow(TRUE);，确保胜利后按钮状态更新。
6. **问题 6**：二维数组释放时出现内存泄漏。
    - 解决：实现 CGameLogic::ReleaseMap()，先遍历释放每行的一维数组，再释放二维数组的首地址，同时将指针置为 NULL。

第三部分：实验小结、收获与体会

一、实验小结

本次实验基于 C++ 和 MFC 框架开发了连连看桌面应用程序，完成了主界面设计、游戏地图生成、消子判断、胜负判断、提示、重排、多模式支持等核心功能。实验严格遵循**面向对象编程思想**和**三层架构设计**，实现了界面与逻辑的解耦，同时充分运用**线性结构**（二维数组、顶点数组）完成了游戏的核心算法设计，达到了数据结构与算法综合应用的实验目的。

实验过程中采用**迭代开发**模式，从基础的工程创建到复杂的消子算法实现，每一步都进行编译、调试和功能验证，确保了项目的稳定性和可扩展性。最终实现的连连看游戏界面美观、交互流畅、功能完整，符合实验要求的所有功能点，且代码具有良好的可读性和可维护性。

二、实验收获

1.编程能力提升：熟练掌握了 C++ 面向对象的编程思想，能够灵活运用类、对象、封装等特性进行模块化设计；掌握了 MFC Dialog 框架的开发流程，包括对话框创建、消息响应、GDI 图形编程、资源管理等核心技术。

2.数据结构应用能力：深入理解了线性结构（数组、结构体）的核心特性，能够将二维数组应用于游戏地图存储，将顶点结构体应用于路径记录，通过数组遍历实现了消子判断、提示、重排等核心算法，提升了数据结构与算法的实际应用能力。

3.问题解决能力提升：在开发过程中遇到了图形绘制、算法逻辑、内存管理等多个问题，通过断点调试、单步执行、查阅资料等方式解决问题，培养了独立分析问题和解决问题的能力，同时掌握了 MFC 开发的常见调试技巧。

4.项目开发思维培养：熟悉了软件项目的**迭代开发流程**，掌握了**需求分析**、**系统设计**、**编码实现**、**调试测试**的完整流程；理解了**三层架构**的设计思想，实现了界面与逻辑的解耦，为后续复杂项目开发奠定了基础。

5.编码习惯养成：在开发过程中注重代码的规范性和可读性，添加了详细的注释，对资源进行了及时释放，避免了内存泄漏，养成了良好的编码习惯和软件工程化思维。

三、实验体会

本次连连看游戏开发实验是对数据结构与算法、C++ 编程、MFC 框架的一次综合应用，让我深刻体会到了**理论与实践结合**的重要性。数据结构中的线性结构（数组、结构体）并非孤立的理论知识，而是可以直接应用于实际项目的核心工具，消子判断、地图存储、路径记录等核心功能的实现，都依赖于线性结构的灵活运用。

同时，**模块化设计**和**分层架构**是提升项目可维护性和可扩展性的关键。本次实验将项目分为界面层、控制层、逻辑层，各层职责明确，界面层只负责绘制和交互，

逻辑层只负责算法实现，控制层协调层间交互，使得后续功能扩展（如添加计时、关卡模式）变得更加简单，只需在对应层级添加代码，无需修改其他层级的逻辑。

此外，**调试能力**是软件开发的核心能力之一。在实验过程中，很多问题并非语法错误，而是算法逻辑或图形绘制的细节问题，通过断点调试和单步执行，能够快速定位问题所在，提高开发效率。同时，及时释放资源（如 DC、位图、二维数组）是保证程序稳定性的重要环节，避免内存泄漏和程序崩溃。

本次实验也让我认识到了自己的不足，例如对 MFC GDI 编程的细节掌握不够深入，对复杂算法（如三条直线连通判断）的设计思路不够清晰。在后续的学习中，我将加强对 MFC 框架和高级算法的学习，同时尝试开发更复杂的软件项目，进一步提升自己的编程能力和项目开发能力。

总体而言，本次实验不仅巩固了数据结构与算法、C++ 编程的核心知识，还培养了我的工程实践能力和问题解决能力，为后续的专业学习和工程开发奠定了坚实的基础。